

Contents

[VIDEO] Roteiro do Vídeo — GS POO 2026/1	1
Sistema Pequii-Sol	1
[TEMPO] 0:00 — ABERTURA (rosto na câmera)	1
[TEMPO] 0:30 — VISÃO GERAL	1
[TEMPO] 1:15 — INTERFACE Operavel	2
[TEMPO] 2:15 — CLASSE ABSTRATA Satellite	2
[TEMPO] 3:30 — SUBCLASSES	2
[TEMPO] 5:00 — CLASSE Main	3
[TEMPO] 7:00 — [DEMO] DEMONSTRAÇÃO AO VIVO	4
Sequência dos testes:	4
» Fale enquanto demonstra:	4
[TEMPO] 9:30 — ENCERRAMENTO (rosto na câmera)	5
[OK] CHECKLIST ANTES DE GRAVAR	5
[DICA] DICAS FINAIS	5
BOA GRAVAÇÃO!	5

[VIDEO] Roteiro do Vídeo — GS POO 2026/1

Sistema Pequii-Sol

Duração alvo: 8 a 10 minutos

[TEMPO] 0:00 — ABERTURA (rosto na câmera)

“Olá, professor Selmini. Meu nome é **[SEU NOME]**, RM **[SEU RM]**, e este é o vídeo da minha Global Solution de Programação Orientada a Objetos com Java.

Vou apresentar o **Sistema de Gestão da Constelação Pequii-Sol**, explicar a arquitetura do projeto, mostrar o código de cada classe e em seguida fazer uma demonstração da execução.”

Mostrar: IDE aberta com a árvore do projeto.

[TEMPO] 0:30 — VISÃO GERAL

“O sistema gerencia três tipos de satélites: **CubeSats**, satélites de **Comunicação** e satélites de **Observação**.

Como todos compartilham atributos básicos (id, nome, altitude e status), usei **herança**.

E como todos precisam reportar o consumo de energia com fórmulas diferentes, criei uma **interface** chamada Operavel.

O projeto está organizado em três pacotes: modelo, operacao e app.”

Mostrar: expandir os pacotes.

[TEMPO] 1:15 — INTERFACE Operavel

Abrir: Operavel.java

“Começando pela interface Operavel. É bem simples: tem apenas o método calcularConsumoEnergia() que retorna um double.

Toda classe que implementar essa interface é obrigada a fornecer a sua própria fórmula de consumo.

A vantagem é o **polimorfismo**: posso tratar qualquer satélite como Operavel sem saber qual tipo é.”

[TEMPO] 2:15 — CLASSE ABSTRATA Satellite

Abrir: Satellite.java

“Esta é a classe-base, declarada como abstract — ou seja, **não pode ser instanciada diretamente**, só serve para ser herdada.

Tem os quatro atributos comuns: id, nome, altitudeKm e status, todos private para garantir o **encapsulamento**.

O acesso é feito por getters e setters.

Reparem que o setter da altitude faz validação: se for menor ou igual a zero, lança IllegalArgumentException. Assim, **nunca vai existir um satélite com altitude inválida em memória**.

O toString() retorna os atributos comuns. As subclasses sobrescrevem para acrescentar os seus.”

Destacar: abstract, private, validação.

[TEMPO] 3:30 — SUBCLASSES

Abrir: CubeSat.java

“A primeira subclasse é o CubeSat. Ela extends Satellite e implements Operavel.

No construtor, chamo super(...) para inicializar os atributos da classe-pai e valido a massa. O atributo próprio é massaKg.

A fórmula segue exatamente o enunciado: $5.0 + \text{massaKg} * 0.8$.

O toString() foi sobrescrito com @Override: chama super.toString() e adiciona massa e consumo. Assim aparece os **dados reais**, não endereço de memória.”

Abrir: SatelliteComunicacao.java

“O SatelliteComunicacao segue a mesma estrutura. O atributo é a largura de banda em Mbps, e a fórmula é $30 + \text{larguraBandaMbps} * 1.5$.”

Abrir: SatelliteObservacao.java

“O SatelliteObservacao tem a resolução em metros. A fórmula é $50 + 1000 / \text{resolução}$.”

Quanto menor a resolução, melhor a precisão e maior o consumo dos sensores. Como divisão por zero ou negativo daria um valor absurdo, o setter rejeita esses casos.”

[TEMPO] 5:00 — CLASSE Main

Abrir: Main.java

“Agora a classe Main, que contém toda a lógica de interação.”

Destacar MENU:

“Aqui o texto do menu, exatamente como o enunciado pediu.”

Destacar ArrayList<Satelite>:

“Esta é a coleção. É um ArrayList<Satelite> — graças ao **polimorfismo**, guardo os três tipos na mesma lista.”

Mostrar while e try/catch:

“No main tenho um while que exibe o menu até o usuário sair.

Tudo dentro de um try/catch que captura três exceções: - NumberFormatException para texto onde era número - IllegalArgumentException para valores inválidos - Um catch genérico para qualquer outra falha

Em **nenhum caso o programa encerra** — sempre volta ao menu, conforme exigido.

Também trato o null que o JOptionPane retorna quando o usuário clica em Cancelar.”

Percorrer cada método:

“**Registrar:** pergunta o tipo, pergunta o ID, e antes de tudo checo se já existe com buscarPorId. Se existir, recuso o cadastro.

Pesquisar por ID: uso buscarPorId e exibo o toString() ou mensagem de não encontrado.

Alterar status: localizo o satélite, peço o novo status, e o método pedirStatus-Valido aceita só operacional, manutenção ou desativado. Aplica toLowerCase para funcionar **independente de maiúscula ou minúscula**, como pede o enunciado.

Excluir: localizo, removo e confirmo.

Listar: percorro com um for-each chamando toString() de cada satélite. Se vazia, exibe 'Nenhum satélite cadastrado.'

Sair: encerra com mensagem de despedida."

[TEMPO] 7:00 — [DEMO] DEMONSTRAÇÃO AO VIVO

"Agora vou executar para mostrar funcionando."

Sequência dos testes:

1. Opção **5** (Listar) — lista vazia → Mostra "Nenhum satélite cadastrado."
2. Opção **1** — cadastrar **CubeSat** - ID: 1 - Nome: Pequi-01 - Altitude: 400 - Status: operacional - Massa: 2
3. Opção **1** — cadastrar **Comunicação** - ID: 2 - Nome: Conecta-BR - Altitude: 550 - Status: OPERACIONAL (em maiúsculo, de propósito!) - Banda: 100
4. Opção **1** — cadastrar **Observação** - ID: 3 - Nome: OlhoAmazonia - Altitude: 700 - Status: manutenção - Resolução: 5
5. Opção **5** — Listar → Mostra os 3 com seus consumos calculados
6. Opção **2** — pesquisar ID 99 → "Não encontrado"
7. Opção **2** — pesquisar ID 2 → Mostra os dados do satélite
8. Opção **3** — alterar status do ID 3 para desativado
9. Opção **1** — tentar cadastrar **ID 1 de novo** → Bloqueia: "ID já existe"
10. Opção **1** — digitar **letra** no ID → "Entrada inválida: digite um número."
11. Opção **1** — cadastrar com **altitude -5** → IllegalArgumentException tratada
12. Opção **1** — Observação com **resolução 0** → Rejeita
13. Opção **3** — status inválido (ex: ligado) → Rejeita
14. Opção **4** — excluir ID 1
15. Opção **5** — Listar (confirma exclusão)
16. Opção **1** — começar e clicar **Cancelar** → Volta ao menu sem erro
17. Opção **6** — Sair

» Fale enquanto demonstra:

"Olha: digitei letras no lugar do número, e o programa **não quebrou** — capturou o NumberFormatException e voltou ao menu."

"Aqui tentei cadastrar com o mesmo ID — **bloqueado**, como o enunciado pediu."

"Digitei o status em MAIÚSCULAS e o sistema aceitou, porque **normalizo com toLowerCase**."

[TEMPO] 9:30 — ENCERRAMENTO (rosto na câmera)

“Esses foram os pontos principais. Resumindo o que apliquei:

- **Herança** com a classe abstrata `Satelite`
- **Interface** `Operavel` para o contrato de consumo
- **Polimorfismo** no `ArrayList<Satelite>`
- **Encapsulamento** com atributos privados e setters validados
- **Tratamento de exceções** em três níveis para garantir que o programa nunca encerre

Obrigado, professor! Qualquer dúvida estou à disposição.”

[OK] CHECKLIST ANTES DE GRAVAR

- ☐ Aumentar a **fonte da IDE** (Ctrl+Shift+= no IntelliJ)
 - ☐ Fechar abas e painéis que não vai usar — tela limpa
 - ☐ Testar o **microfone** antes
 - ☐ Configurar gravador (OBS / Win+G / QuickTime)
 - ☐ **Webcam no canto** — seu rosto precisa aparecer
 - ☐ Ensaiar **1 vez** antes de gravar
 - ☐ Upload no **YouTube como “não listado”** e enviar só o link
 - ☐ **Prazo: hoje, 12/06, 23h55**
-

[DICA] DICAS FINAIS

- **Não leia palavra por palavra** — use como guia mental
 - Errar e se corrigir naturalmente é melhor que tom robótico
 - Se errar feio, respire e refaça o trecho
 - Fala com calma, sem pressa de terminar
 - Olhe para a câmera nos momentos de “rosto na câmera”
-

BOA GRAVAÇÃO!